

INCIDENT DOOMSDAY



Student Workbook

Build the tool, then investigate.

DAYS 1A + 1B

BUILD

Variables, arrays, for loops, while loops. Write `evidence_processor.sh`.

DAY 2

INVESTIGATE

Search, filter and correlate records across six logs.

DAY 3

DEFEND

Timeline, incident report and evidence-based debrief.

NAME

PERIOD

DATE

FICTION NOTICE

This is an invented incident involving a real movie title. Every event, person, account, system and evidence record is fictional. This activity makes no claim about a real leak. All website names use reserved .example domains; no website visits are needed.

DAY 1A

Variables and arrays

Build the tool. Keep the case open.

50 MINUTES

MISSION Today you build the tool. Read **Case_Briefing** first and confirm your terminal is in **STUDENT_PACKAGE**.

1 Why automate?

5 MIN

```
cat logs/login.log
cat logs/downloads.log
```

What changes between the two commands?

If you had 100 files, what work would repeat?

2 One variable

8 MIN

HEADS UP

Predict each output **before** you run it. Assignment has **no spaces** around **=**.

```
MOVIE="Avengers: Domsday"
echo MOVIE
echo "$MOVIE"
```

Command	Your prediction	Actual output
echo MOVIE		
echo "\$MOVIE"		

KEY IDEA

In **echo MOVIE**, MOVIE is literal text. In **echo "\$MOVIE"**, the **\$** asks Bash for the stored value. Explain that difference to a partner.

Change MOVIE's value and run the same echo command. Why does the output change?

3 Build a path

10 MIN

```
LOG_DIR="logs"
FILE="login.log"
echo "$LOG_DIR/$FILE"
cat "$LOG_DIR/$FILE"
```

Predict the path that will print:

Which command prints a path, and which prints file contents?

Change only FILE to downloads.log and run the same cat command. Explain why it works:

4 Store a collection

12 MIN

```
LOG_FILES=("login.log" "file_access.log" "downloads.log" "usb_activity.log"
           "network.log" "security.log")
echo "${LOG_FILES[0]}"
echo "${LOG_FILES[2]}"
echo "${LOG_FILES[@]}"
```

Predict all three outputs:

Fill in the index / value table for your array:

Index	Value
0	
1	
2	
3	
4	
5	

What does the @ request?

KEY IDEA

The quoted array expansion supplies each element as a **separate argument**. **echo** then displays them on one line, separated by spaces.

```
for file in login.log downloads.log security.log
do
  echo "$file"
done
```

Round	Value of file
Round 1	
Round 2	
Round 3	

Is "file" a special Bash keyword, or a variable we chose?

THINK

Exit check: Write one variable assignment and retrieve its value. Then explain why an array helps organize filenames.

Your assignment + retrieval:

Why an array helps:

DAY 1B

Loops and the evidence processor

50 MINUTES

Connect arrays to loops, then nest a while loop inside.

MISSION Turn six filenames into one script that reads every line of every log.

6 Connect array to loop

8 MIN

Recreate **LOG_DIR** and **LOG_FILES** in your script. Right after those assignments, add:

```
for file in "${LOG_FILES[@]}"
do
    echo "$LOG_DIR/$file"
done
```

How many paths should print?

What changes on each repetition?

7 Open each file

7 MIN

Replace the echo inside your loop with **cat "\$LOG_DIR/\$file"**. Add **echo "EVIDENCE FILE: \$file"** before the cat.

How many cat statements did you write?

How many times was that statement executed?

Test this **separate one-file example** before nesting it:

```
LOG_DIR="logs"
FILE="login.log"

while IFS= read -r line
do
    echo "$line"
done < "$LOG_DIR/$FILE"
```

Piece	What it does
read	Stores the current line in the variable line .
IFS=	Preserves leading and trailing whitespace.
-r	Preserves backslashes.
< file	Sends the file into the loop as its input.
Quotation marks	Keep paths and text together.

What does line contain?

Where does read get its input?

HEADS UP

For these newline-terminated files this form reads every line. An advanced, more robust version also checks **|| [[-n "\$line"]]** to keep a final line that has no trailing newline.

In `evidence_processor.sh`, build from these pieces. **Do not copy a completed solution.**

- Start with `#!/usr/bin/env bash`.
- Set a case-name variable and `LOG_DIR`.
- Define the six-element `LOG_FILES` array.
- Begin the `for` loop; print the current filename.
- Inside it, begin the `while` loop; print the current line.
- Close the while loop and redirect the current file into it.
- Close the for loop. Run `bash evidence_processor.sh`.

SKETCH: draw two labeled boxes. Outside selects FILE. Inside processes LINES.

When does Bash return to select the next file?

```
cp extension_evidence/manifest.log logs/manifest.log
```

Add its filename to your array **without changing either loop**. Predict, then verify.

Your prediction:

Verified result:

What makes this scalable?

11 Exit ticket

7 MIN

THINK

Explain each item in your own words. Submit your script and a short explanation.

Variable vs. array

What "\${LOG_FILES[@]}" does

The changing value of file

The role of line

Why the while loop is inside the for loop

CASE STATUS

OPEN

DAY 2

Search, filter, and correlate

50 MINUTES

Follow the evidence across all six logs.

MISSION Change **LOG_DIR** to **logs_expanded**. Do **not** add manifest.log: the expanded folder has the original six files.

12 Learn a search

8 MIN

```
grep -n 'DOOMSDAY_FINAL_MASTER_4K.mov' logs_expanded/file_access.log
grep -n 'ALERT' logs_expanded/security.log
grep -i 'login_failed' logs_expanded/login.log
```

Flag	Meaning
-n	Displays the source line number.
-i	Ignores letter case.

Filename : line	Result you copied

Is an alert alone proof? Explain:

13 Count and summarize

7 MIN

```
grep 'event=LOGIN_FAILED' logs_expanded/login.log | wc -l
grep -h 'event=LOGIN_FAILED' logs_expanded/login.log | sort | uniq -c
```

KEY IDEA

| passes output onward. **wc -l** counts matching lines. **sort** groups identical text so **uniq -c** can count identical records. It does **not** automatically count per user when timestamps differ.

How many failed-login records exist?

Why are the individual uniq counts all 1?

14 Follow an identifier

15 MIN

Choose a master-file access record. Search its session across all six logs:

```
grep -n 'session=YOUR_SESSION' logs_expanded/*.log
```

WATCH OUT

Replace **YOUR_SESSION** with a value you actually found. Track **asset_id**, **transfer**, **bytes**, **hash**, **time** and **destination**. Search tickets and the destination allowlist **before** drawing conclusions.

UTC timestamp	Source file : line	Account / session	Observed event	Why it matters

15 Test alternatives

10 MIN

Find evidence explaining each activity. Record its approval or explanation, and **one limitation** of your interpretation.

Activity	Approval / explanation	Limitation of my interpretation
Editor's late-night access		
IT USB connection		
VFX failed logins		
Marketing upload		

16 Optional condition

5 MIN

Inside your while loop, try:

```
if [[ "$line" == *"ALERT"* ]]
then
    echo "FLAG: $line"
fi
```

HEADS UP

If you keep your original echo, every line prints, and alert lines print again with FLAG. Remove the original echo only if you want alert-only output.

Why must this condition be inside the while loop?

17 Exit check

5 MIN

Which account deserves follow-up?

Log 1: record	Log 2: record	Log 3: record

What do you still need to establish who operated the account?

DAY 3

Defend your conclusion

Report what you can prove, and say what you cannot.

50 MINUTES

MISSION Use **Incident_Report_Template**. Do not identify a person solely from an account name.

1

Timeline

10 min

2

Incident report

15 min

3

Partner review

15 min

4

Debrief

10 min

Keep these three things separate in your report:

OBSERVED FACT

What the records directly show.

INFERENCE

What you reason from the facts.

UNCERTAINTY

What you still cannot establish.

A

Chronological timeline

10 MIN

UTC time	Source (file : line)	What happened	Fact / Inference

THINK

Partner review checklist: Does every claim cite a record? Are facts, inferences and uncertainty labeled? Does any sentence name a person based only on an account name?

QUICK REF

Syntax at a glance

Keep this page beside your terminal.

1 PAGE

MISSION Everything you used in this workbook, in one place.

Task	Syntax	Notes
Variable	<code>NAME="value"</code>	No spaces around the equals sign.
Read a variable	<code>echo "\$NAME"</code>	The dollar sign asks for the stored value.
Array	<code>ARR=("a" "b" "c")</code>	Index starts at 0.
One element	<code>"\${ARR[2]}"</code>	Gets the item at index 2.
Every element	<code>"\${ARR[@]}"</code>	Each item passed as a separate argument.
for loop	<code>for x in "\${ARR[@]}; do ... done</code>	Repeats once per item. x is a name you choose.
while + read	<code>while IFS= read -r line; do ... done < file</code>	Reads one line at a time from the file.
Search	<code>grep -n 'text' file</code>	-n shows line numbers; -i ignores case; -h hides filenames.
Search many files	<code>grep -n 'text' logs_expanded/*.log</code>	The star matches every .log file.
Count lines	<code>grep ... wc -l</code>	The pipe passes output to the next command.
Count repeats	<code>grep ... sort uniq -c</code>	Sort first so identical lines sit together.
Condition	<code>if [["\$line" == *"ALERT"*]]; then ... fi</code>	Must sit inside the loop it tests.